Solar BI

Solución de Business Intelligence

Sebastián Pineda Álvarez

Esneyder Briñez Gomez

Docente

Ramiro Grisales Montoya

Ingeniería de Software

Institución Universitaria Pascual Bravo

Medellín
2026

INSTITUCIÓN UNIVERSITARIA PASCUAL BRAVO
Estrategia de Articulación EnContexto · Departamento de Sistemas Digitales

Trabajo de consulta en parejas

SolarBI Pascual: Power BI y Grafana, gobernanza de datos, automatización ETL e IoT

Inteligencia de Negocios · Grupo 01 · Semestre 2026-II · Docente: Ramiro Grisales Montoya

1. Propósito

En SolarBI el cuadro de mando final se construye en Power BI. Sin embargo, en la industria el monitoreo de plantas y equipos suele hacerse con Grafana. En esta consulta compararán las dos herramientas sobre los mismos datos y darán sus primeros pasos en tres temas que sostienen el proyecto: gobernanza de datos, automatización de procesos ETL e Internet de las Cosas (IoT), trabajando con control de versiones en GitHub.

> **Enfoque del Grupo 01:** monitoreo operativo. Su práctica se centra en la potencia en el tiempo y en la detección de fallas.

2. Ficha del trabajo

Campo	Contenido
Curso / Grupo	Inteligencia de Negocios · Grupo 01
Modalidad	Parejas (dos integrantes). Una sola entrega por pareja.
Tiempo disponible	Dos (2) días a partir de la publicación en Google Classroom.
Fecha límite	_____
Valor	_____
Articulación	Proyecto de Aula SolarBI Pascual. Prepara el Entregable III (esquema estrella y flujo ETL, semana 12) y el Entregable V (cuadro de mando, semana 16).
Entrega	Un archivo PDF central + repositorio en GitHub con los archivos de soporte.

Indicadores de resultado de aprendizaje que moviliza: U2-a (lago de datos), U2-b (calidad de datos), U2-d (flujos ETL), U3-d (indicadores de desempeño), U3-e (cuadros de mando) y U2-e (responsabilidad frente a la información).

3. Parte A · Consulta

Respondan cada pregunta con sus propias palabras, en máximo 120 palabras, aplicándola al caso SolarBI e indicando la fuente. Las tablas, los diagramas y los fragmentos de código no cuentan en el límite.

Bloque 1 · Power BI y Grafana

D1. ¿Qué es Grafana? Describan sus componentes: data sources, panels, dashboards, variables y alerting. Diferencien Grafana OSS de Grafana Cloud.

Grafana es una plataforma de código abierto para visualización y monitoreo técnico en tiempo real.

Grafana OSS (Open Source Software) es la edición gratuita, mientras que Grafana Cloud es gestionada en la nube (SaaS)

- **Data Sources:** Conexiones que consultan bases de datos para extraer métricas o registros.
- **Panels:** Bloques básicos que combinan una consulta con una visualización gráfica.
- **Dashboards:** Conjunto de paneles organizados para dar una visión global.
- **Variables:** Filtros dinámicos que actualizan consultas y títulos según la selección.
- **Alerting:** Mecanismo automático que evalúa datos y envía notificaciones al detectar fallos.

D2. Elaboren una tabla comparativa Power BI vs. Grafana con mínimo 8 criterios: propósito principal, usuario típico, conexión a los datos, modelo semántico y lenguaje (DAX frente a la consulta del origen), actualización y tiempo real, alertas, licenciamiento y costo, y control de acceso.

Criterio	Power BI	Grafana
1. Propósito principal	Inteligencia de negocios, analítica estratégica/histórica y reportes ejecutivos.	Monitoreo técnico operacional, métricas de infraestructura e IoT en tiempo real.
2. Usuario típico	Analistas de negocio, gerentes, directivos y tomadores de decisiones.	Operadores de planta, ingenieros DevOps, administradores de red y mantenimiento.
3. Conexión a los datos	Conexión mediante conectores nativos, Import Mode (en memoria) o DirectQuery.	Consulta directa a bases de datos de series temporales (TimescaleDB, InfluxDB, PostgreSQL).
4. Modelo semántico y lenguaje	Modelo dimensional en estrella; usa **DAX** para medidas y **Power Query (M)** para ETL.	Consultas directas en el lenguaje del origen (ej. **SQL/PromQL**); no crea modelos complejos.
5. Actualización y tiempo real	Programada por lotes (ej. 8 veces/día en Pro) o Near Real-Time (streaming datasets).	Tiempo real nativo (*Live streaming*) con refrescos continuos de segundos.

6. Alertas	Alertas de datos basadas en umbrales en tarjetas de paneles (*Power BI Service*).	**Grafana Alerting**: motor robusto con evaluación de condiciones complejos y *contact points*.
7. Licenciamiento y costo	Pro/Premium por usuario o capacidad. Versión Desktop gratuita.	**OSS**: Gratuita auto-hospedada. **Cloud**: Plan *freemium* y pago por uso (*SaaS*).
8. Control de acceso	Control de acceso por roles en Power BI Service y **RLS** (Row-Level Security).	Organizaciones, equipos, roles (Admin/Editor/Viewer) y permisos por carpeta/dashboard.

Aplicado al caso SolarBI, Power BI se utiliza para evaluar la producción acumulada de energía, métricas de rendimiento (Yield, Performance Ratio) y retorno económico para la toma de decisiones. Por su parte, Grafana se conecta directamente a la base de datos (PostgreSQL/TimescaleDB) para supervisar la potencia instantánea y la irradiancia en tiempo real, emitiendo alertas inmediatas si un inversor falla.

D3. Diferencien un dashboard operativo, uno analítico y uno estratégico. Clasifiquen cada KPI mínimo del proyecto (energía, yield y Performance Ratio, ahorro, disponibilidad y alarmas, porcentaje de datos válidos) e indiquen en cuál de las dos herramientas lo publicarían y por qué.

Un dashboard operativo monitorea actividades en tiempo real para acciones inmediatas; uno analítico examina tendencias históricas y causas raíz; y uno estratégico mide el rendimiento global a largo plazo para la alta dirección.

KPI Mínimo	Tipo de Dashboard	Herramienta de publicación	Justificación

Disponibilidad y alarmas	Operativo	Grafana	Monitoreo en tiempo real y detección rápida de fallas técnicas en los inversores.
Porcentaje de datos válidos	Operativo / Analítico	Grafana	Permite verificar la calidad de la telemetría en tiempo real y detectar fallas en sensores.
Energía	Analítico	Power BI	Permite comparar la generación diaria/mensual e identificar patrones de producción.
Yield y Performance Ratio (PR)	Analítico	Power BI	Evalúa la eficiencia del sistema frente a la radiación mediante modelos de datos.
Ahorro ($)	Estratégico	Power BI	Mide el impacto económico del proyecto para la toma de decisiones directivas.

D4. Grafana con PostgreSQL: ¿cómo debe ser una consulta de series de tiempo y para qué sirven las macros `$__timeFilter` y `$__timeGroup`? Escriban la consulta de la potencia promedio por hora.

Una consulta de series de tiempo en Grafana debe devolver una columna de tiempo (time o time_column), la variable a graficar como valor numérico y opcionalmente el nombre de la métrica. La macro $__timeFilter aplica automáticamente el filtro del rango temporal seleccionado en la interfaz. La macro $__timeGroup agrupa y redondea las marcas de tiempo según el intervalo del gráfico para hacer agregaciones

```
SELECT
  $__timeGroup(ts, '1h') AS time,
  AVG(p_ac_kw) AS "Potencia Promedio (kW)"
FROM silver.lectura_5min
WHERE $__timeFilter(ts)
GROUP BY 1
ORDER BY 1;
```

D5. Alertas: comparen Grafana Alerting (alert rules y contact points) con las alertas de datos del servicio de Power BI. Propongan una regla de alerta útil para la planta y el canal por el que se notificaría.

Grafana Alerting evalúa consultas complejas mediante alert rules y las envía a múltiples contact points (Slack, correo, Webhooks) de forma nativa e inmediata. Las alertas de Power BI evalúan solo tarjetas o medidores en paneles compartidos (Power BI Service) al actualizar los datos, con notificaciones limitadas (correo o Power Automate).

Regla de alerta propuesta: Si entre las 09:00 y las 15:00 horas la irradiancia es mayor a 200 W/m² pero la potencia activa (p_ac_kw) es menor o igual a 0 kW durante más de 10 minutos, se genera una alerta crítica.

Canal de notificación: Canal de Telegram u Operaciones en Slack del equipo de mantenimiento.

Bloque 2 · Gobernanza de datos (data governance)

G1. Definan data governance y diferéncienla de data management y de data quality. Tomen como referencia el marco DAMA-DMBOK.

Según el marco DAMA-DMBOK, la Gobernanza de Datos (Data Governance) es el ejercicio de autoridad, control y toma de decisiones sobre la gestión de los activos de datos. Define las políticas, estándares, roles y responsabilidades de la organización.

- Diferencia con Data Management: La gobernanza establece las reglas, derechos de decisión y políticas (el "qué" y el "quién"), mientras que la gestión de datos (Data Management) ejecuta la parte operativa y técnica para almacenar, transformar, integrar y asegurar dichos datos (el "cómo").
- Diferencia con Data Quality: La calidad de datos es una disciplina técnica dentro del manejo de datos que mide dimensiones específicas (precisión, completitud, consistencia), mientras que la gobernanza fija los umbrales de calidad aceptables para el negocio.

G2. Expliquen los roles data owner, data steward y data custodian. Relaciónenlos con los roles del equipo en el proyecto (data engineer, data modeler, BI analyst y product owner).

En gobernanza de datos existen tres roles fundamentales:

- Data Owner (Dueño de Datos): Responsable del negocio que rinde cuentas sobre un dominio de datos, aprueba accesos y fija definiciones.
- Data Steward (Custodio Funcional): Garantiza el cumplimiento de políticas de calidad, metadatos y uso correcto en el día a día.
- Data Custodian (Custodio Técnico): Administra la arquitectura técnica, almacenamiento, infraestructura y seguridad física de la base de datos.

G3. El contrato de datos (data contract) del proyecto como instrumento de gobernanza: ¿qué debe incluir (esquema, unidades, frecuencia, responsable, niveles de calidad)? ¿Qué le pasa al cuadro de mando si la fuente cambia un campo sin avisar?

El contrato de datos es un acuerdo formal y estructurado (usualmente en YAML o JSON) entre los productores de datos (ej. simulación/inversores IoT) y los consumidores (analistas/BI). Debe incluir:

1. Esquema: Nombre de columnas y tipos de datos (ts, dispositivo_id, p_ac_kw).
2. Unidades: Especificación explícita (kW, W/m², °C).
3. Frecuencia: Periodicidad de llegada de lectura (cada 5 minutos).
4. Responsable: Nombre del productor de datos o equipo asignado.
5. Niveles de calidad (SLA): Porcentaje mínimo de datos válidos (ej. > 95%), límites físicos e idempotencia.

Consecuencia de un cambio no anunciado: Si la fuente cambia o renombra un campo sin avisar, el flujo ETL fallará en la ingesta o los reportes de Power BI y Grafana arrojarán errores de carga, rompiendo los dashboards y desajustando las métricas calculadas en DAX/SQL.

G4. Seguridad a nivel de fila (row-level security, RLS) en Power BI: roles y filtros DAX. Den el ejemplo de un responsable que solo ve su sitio. ¿Cuál es el mecanismo equivalente de control de acceso en Grafana (organizaciones, equipos y permisos por carpeta o dashboard)?

En Power BI, la seguridad a nivel de fila (Row-Level Security - RLS) restringe el acceso a los datos según el usuario. Se crean roles en Power BI Desktop aplicando filtros DAX sobre las tablas dimensionales.

En Grafana, el control de acceso equivalente no filtra filas mediante consultas DAX, sino que organiza la seguridad jerárquicamente por:

- Organizaciones: Aislamiento total de entornos.
- Equipos (Teams): Grupos de usuarios asignados.
- Permisos por Carpeta/Dashboard: Roles (Admin, Editor, Viewer) aplicados sobre carpetas específicas de tableros para limitar la visión del usuario.

G5. Ley 1581 de 2012 (protección de datos personales en Colombia): ¿los datos de SolarBI son personales? ¿Qué cambiaría si el tablero mostrara qué operador atendió cada alarma? Fijen la postura del equipo frente al manejo responsable de la información.

Los datos técnicos puros de telemetría de SolarBI (potencia, irradiancia, temperatura) no son datos personales, pues miden variables de máquinas e infraestructura física.

Sin embargo, sí se convertirían en datos personales (específicamente datos de desempeño laboral o identificación) si el tablero añade qué operador atendió cada alarma. Bajo la Ley 1581 de 2012 de Colombia, esto requeriría autorización previa, explícita e informada del trabajador, aviso de privacidad y políticas de tratamiento de datos personales.

Postura ética del equipo: Apostamos por la privacidad desde el diseño. Los tableros monitorearán la eficiencia técnica y el estado de la planta sin exponer métricas punitivas individuales de los trabajadores, promoviendo el manejo transparente, seguro y responsable de la información.

Bloque 3 · Automatización de procesos ETL

E1. Diferencien ETL de ELT y ubiquen cada paso en las capas Bronze, Silver y Gold del proyecto. ¿Power Query es una herramienta ETL? Argumenten.

En ETL, los datos se transforman en un servidor intermedio antes de cargarse al destino. En ELT, los datos crudos se cargan directamente en el destino (Data Lake/Warehouse) y la transformación se realiza mediante el motor de la base de datos.

- Bronze (ELT/ETL): Ingesta y almacenamiento de datos crudos simulados sin modificar (telemetria.csv).

- Silver (ETL): Limpieza, validación de reglas de calidad y guardado en silver.lectura_5min.
- Gold (ELT): Agregación diaria de la energía producida para la tabla fact_energia_dia.

E2. Comparen carga completa (full load) y carga incremental. ¿Qué significa que una carga sea idempotente y cómo se logra en `fact_energia_dia` con su clave primaria (`fecha_key`, `dispositivo_key`) y un UPSERT?

La carga completa borra y recarga todo el dataset en cada ejecución, mientras que la carga incremental procesa únicamente los registros nuevos o modificados desde la última actualización.

Una carga es idempotente si ejecutar el proceso múltiples veces con los mismos datos genera exactamente el mismo resultado final sin duplicar registros. En fact_energia_dia, esto se logra definiendo una clave primaria compuesta (fecha_key, dispositivo_key) y aplicando una sentencia UPSERT (INSERT ... ON CONFLICT DO UPDATE en PostgreSQL). Si la fila existe, actualiza sus valores; si no existe, la inserta.

E3. Elaboren una tabla comparativa de tres formas de automatizar un flujo: cron (o el Programador de tareas de Windows), Apache Airflow y la actualización programada de Power BI. Incluyan qué es, qué automatiza cada una y cuándo conviene.

Criterio	**Cron / Programador de Tareas**	**Apache Airflow**	**Actualización Programada Power BI**
¿Qué es?	Programador de tareas nativo del sistema operativo (Linux/Windows).	Plataforma de orquestación de flujos de trabajo basada en código Python (DAGs).	Servicio gestionado en la nube (*Power BI Service*) mediante *Data Gateways*.
¿Qué automatiza?	Scripts individuales (ej. python run_etl.py) ejecutados a horas fijas.	Flujos ETL complejos con dependencias, reintentos, alertas y múltiples tareas.	Refresco de datos e importación de modelos en reportes publicados.
¿Cuándo conviene?	Tareas sencillas, scripts locales o flujos sin dependencias complejas.	Entornos empresariales con canalizaciones masivas, monitoreo e interdependencias.	Actualizar la capa de presentación (*dashboards*) para el usuario final.

E4. Query folding en Power Query: ¿qué es, por qué mejora el rendimiento y cómo se verifica si un paso se está plegando hacia PostgreSQL?

El Query Folding (plegado de consultas) es la capacidad de Power Query de traducir los pasos de transformación definidos en la interfaz a una única consulta nativa en el lenguaje de la fuente de origen (como SQL en PostgreSQL).

¿Por qué mejora el rendimiento?

Evita transferir volúmenes masivos de datos innecesarios a través de la red, delegando el esfuerzo de filtrado y procesamiento al servidor de la base de datos origen.

¿Cómo se verifica?

En Power Query Editor, haz clic derecho sobre un paso en el panel Pasos Aplicados. Si la opción "Ver la consulta nativa" (View Native Query) está habilitada, el paso se está plegando hacia PostgreSQL; si aparece deshabilitada o en gris, el procesamiento se realiza en la máquina local.

Bloque 4 · Internet de las Cosas (IoT)

I1. Describan la arquitectura IoT por capas: dispositivo, gateway (edge), red o broker, almacenamiento y aplicación. Dibujen cómo viaja una lectura del inversor hasta llegar a un visual del tablero, indicando en qué capa (Bronze, Silver, Gold) queda en cada momento.

La arquitectura IoT se divide en:

1. Dispositivo: Sensores e inversores que miden variables físicas.
2. Gateway (Edge): Concentrador de lecturas que las transmite a la red.
3. Broker / Red: Enruta los mensajes (protocolo MQTT/HTTP).
4. Almacenamiento: Almacena la telemetría en la base de datos time-series.
5. Aplicación: Presentación visual e interacción con el usuario.

- Bronze: Telemetría cruda recibida en PostgreSQL/TimescaleDB.
- Silver: Datos limpios y estructurados en silver.lectura_5min.
- Gold: Datos agregados en dwh.fact_energia_dia para tableros.

I2. Expliquen MQTT: modelo publish/subscribe, broker, topic y niveles de QoS (0, 1 y 2). Propongan la jerarquía de topics de la planta, por ejemplo `pascualbravo/solar/<sitio>/<dispositivo>/<variable>`.

MQTT es un protocolo de mensajería liviano basado en un modelo Publish/Subscribe. El Broker es el servidor central que recibe mensajes de los publishers y los distribuye a los subscribers según el Topic (dirección jerárquica del mensaje).

Niveles de QoS (Quality of Service):

- QoS 0 (At most once): Entrega no garantizada ("dispara y olvida").
- QoS 1 (At least once): Garantiza la entrega al menos una vez (puede duplicar).
- QoS 2 (Exactly once): Entrega exactamente una vez mediante confirmación de 4 pasos.

Jerarquía de topics propuesta:

pascualbravo/solar/bloque21/inversor_01/telemetria

pascualbravo/solar/bloque21/inversor_01/alarmas

I3. Describan una pila abierta típica de monitoreo IoT: Mosquitto, Telegraf, una base de series de tiempo (InfluxDB o TimescaleDB) y Grafana. ¿Qué papel cumple cada componente y dónde encajaría PostgreSQL?

En un ecosistema abierto de monitoreo IoT:

- Mosquitto: Broker MQTT que recibe y enruta la telemetría de los sensores en tiempo real.
- Telegraf: Agente que se suscribe al broker, recolecta las métricas y las transforma.
- Base Time-Series (InfluxDB / TimescaleDB): Guarda y optimiza lecturas temporales con índices de tiempo.
- Grafana: Capa de visualización que consulta la base de datos en tiempo real.

Rol de PostgreSQL: PostgreSQL (con la extensión TimescaleDB) actúa como el motor de almacenamiento principal de series de tiempo para telemetría, al tiempo que guarda las tablas relacionales y el modelo dimensional de Power BI.

I4. Procesamiento por lotes (batch) frente a flujo continuo (streaming). La telemetría llega cada 5 minutos y `fact_energia_dia` tiene granularidad diaria: ¿qué se gana y qué se pierde al agregar? ¿Qué herramienta atiende mejor cada granularidad?

El procesamiento en Streaming analiza la telemetría continua evento por evento (cada 5 min), mientras que el procesamiento Batch agrupa datos en bloques de tiempo (diario).

Al agregar a nivel diario (fact_energia_dia):

- Se gana: Consultas ultra rápidas en Power BI, ahorro de almacenamiento y dashboards estratégicos limpios.
- Se pierde: Detalle intradiario, picos instantáneos de irradiancia y capacidad de detectar fallas operativas de minutos.

Herramienta recomendada: Grafana atiende mejor la granularidad fina en Streaming (lecturas de 5 min en tiempo real), mientras que Power BI atiende mejor la granularidad agregada en Batch para análisis analítico/estratégico.

Bloque 5 · Control de versiones

V1. Diferencien Git de GitHub y definan repository, commit, branch y pull request. Un archivo `.pbix` es binario: ¿qué problema genera en Git y qué resuelve el formato Power BI Project (`.pbip`)? ¿Cómo se versiona un dashboard de Grafana?

Git es un sistema de control de versiones distribuido local, mientras que GitHub es una plataforma en la nube para alojar y colaborar en repositorios Git.

- Repository: Espacio donde se almacena el proyecto y su historial.
- Commit: Captura del estado de los archivos en un momento dado.
- Branch: Línea de desarrollo independiente.
- Pull Request: Solicitud para revisar e integrar cambios de una rama a otra.

Archivo .pbix vs. .pbip:

El archivo .pbix es binario, lo que impide ver diferencias línea por línea (diffs) o fusionar cambios (merges) en Git. El formato Power BI Project (.pbip) soluciona esto guardando el informe y el modelo en archivos de texto estructurado (TMDL y JSON), permitiendo un versionado nativo.

Versionado en Grafana:

Los tableros de Grafana se exportan como archivos JSON para versionarse directamente en Git o se gestionan de forma nativa como código mediante herramientas como Terraform o el proveedor de Kubernetes.

4. Parte B · Práctica guiada: los mismos datos en dos herramientas

Construirán un flujo pequeño pero completo: datos crudos de un dispositivo simulado, limpieza con reglas de calidad, carga en PostgreSQL y dos visualizaciones sobre esa misma base, una en Power BI y otra en Grafana.

Paso 1 · Bronze: datos crudos

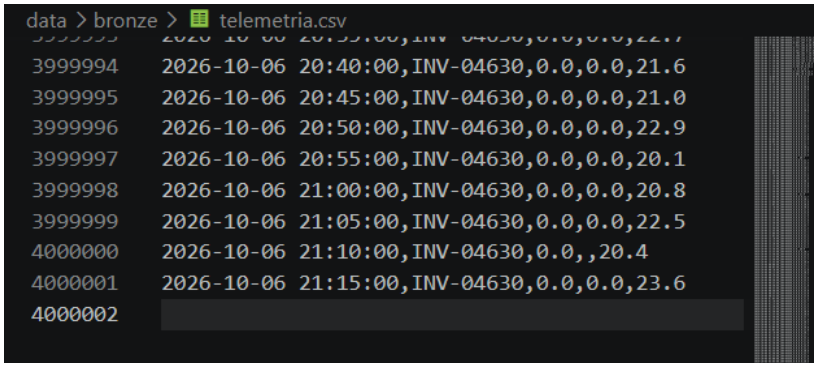
Usen el dataset del contrato de datos del curso o, si lo prefieren, este simulador base (`etl/simulador.py`), que genera 3 días de telemetría de un inversor de 5 kWp con anomalías inyectadas. El archivo de Bronze no se modifica.

```
import csv, math, os, random
from datetime import datetime, timedelta

inicio = datetime(2026, 10, 5, 0, 0)
os.makedirs("data/bronze", exist_ok=True)

with open("data/bronze/telemetria.csv", "w", newline="", encoding="utf-8") as f:
    w = csv.writer(f)
    w.writerow(["ts", "dispositivo_id", "p_ac_kw", "irradiancia_wm2", "temp_modulo_c"])
    for i in range(3 * 288):                          # 3 dias, cada 5 min
        ts = inicio + timedelta(minutes=5 * i)
        sol = max(0.0, math.sin(math.pi * (ts.hour + ts.minute / 60 - 6) / 12))
        irr = round(1000 * sol * random.uniform(0.7, 1.0), 1)
        p_ac = round(5.0 * irr / 1000 * random.uniform(0.80, 0.90), 3)
        fila = [ts.isoformat(sep=" "), 1, p_ac, irr, round(22 + 30 * sol, 1)]
        r = random.random()
        if r < 0.02:
            fila[2] = -1                              # anomalia: potencia negativa
        elif r < 0.04:
            fila[3] = ""                              # anomalia: dato faltante
        w.writerow(fila)
        if r > 0.98:
            w.writerow(fila)                          # anomalia: fila duplicada
```

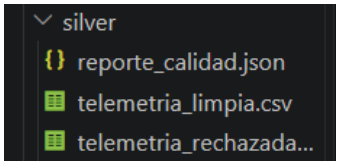
- Generación de registros (telemetria.csv) a partir de archivo ***simulador.py***



Paso 2 · Silver: reglas de calidad

Con Python (pandas) o con SQL, apliquen mínimo tres reglas de calidad: rango físico válido, duplicados y datos faltantes. Reporten filas leídas, filas rechazadas por cada regla, filas válidas y el porcentaje de datos válidos.

- Reglas definidas en archivo *silver.py* y generación de reportes en carpeta /silver



Paso 3 · Gold: carga en PostgreSQL

- Carguen las lecturas limpias de 5 minutos en una tabla `silver.lectura_5min`.

```
[2/4] Cargando lecturas de 5 minutos...
Lecturas cargadas: 3,760,806
```

```
Query  Query History
1  SELECT COUNT(*) FROM silver.lectura_5min;
Data Output  Messages  Notifications
```

	count bigint
1	3760806

```
[3/4] Calculando energia diaria...
Resúmenes diarios generados: 13,889
```

```
Query  Query History
1  SELECT * FROM dwh.fact_energia_dia ORDER BY fecha, dispositivo LIMIT 20;
Data Output  Messages  Notifications
```

	fecha [PK] date	dispositivo [PK] text	energia_kwh double precision	pct_datos_validos double precision
1	2026-10-05	INV-00001	26.854416666666673	95.48611111111111
2	2026-10-05	INV-00002	25.59325000000001	94.44444444444444
3	2026-10-05	INV-00003	25.20458333333332	92.36111111111111
4	2026-10-05	INV-00004	25.61583333333333	93.75
5	2026-10-05	INV-00005	26.291166666666676	93.40277777777777
6	2026-10-05	INV-00006	26.188666666666652	95.83333333333333
7	2026-10-05	INV-00007	25.3885	92.36111111111111
8	2026-10-05	INV-00008	25.76208333333333	94.44444444444444
9	2026-10-05	INV-00009	25.129416666666653	94.44444444444444
10	2026-10-05	INV-00010	26.89066666666668	94.79166666666667
11	2026-10-05	INV-00011	25.349000000000004	92.70833333333333
12	2026-10-05	INV-00012	25.548833333333334	92.70833333333333
13	2026-10-05	INV-00013	26.164499999999986	93.75
14	2026-10-05	INV-00014	25.504333333333328	93.05555555555556
15	2026-10-05	INV-00015	25.529333333333327	94.79166666666667
16	2026-10-05	INV-00016	25.495166666666677	93.05555555555556
17	2026-10-05	INV-00017	25.077416666666668	92.70833333333333
18	2026-10-05	INV-00018	27.134333333333313	94.79166666666667
19	2026-10-05	INV-00019	26.291000000000018	95.13888888888889
20	2026-10-05	INV-00020	26.107583333333324	94.09722222222223

- Carguen el resumen diario en una tabla con la estructura de `dwh.fact_energia_dia` (como mínimo: fecha, dispositivo, `energia_kwh` y `pct_datos_validos`). Recuerden que la energía de cada intervalo es la potencia por 5/60 de hora.

```
def construir_gold(cur):
    print("\n[3/4] Calculando energia diaria...")

    cur.execute("""
        INSERT INTO dwh.fact_energia_dia (
            fecha,
            dispositivo,
            energia_kwh,
            pct_datos_validos
        )
        SELECT
            ts::date AS fecha,
            dispositivo_id AS dispositivo,

            -- Energia = potencia media de cada intervalo
            -- por 5/60 horas. Se suman los intervalos.
            SUM(p_ac_kw * (5.0 / 60.0)) AS energia_kwh,
```

- Dejen todo en un solo punto de entrada (`etl/run_etl.py`) que se ejecute con un único comando. Ejecútenlo dos veces y demuestren que los conteos no cambian (carga idempotente).

Primera ejecución

```
[4/4] Carga completada
Total Silver: 3,760,806
Total Gold: 13,889

ETL finalizado correctamente.
viernes, 9 de octubre de 2026 8:45:18 p. m.
```

Segunda ejecución

```
[4/4] Carga completada
Total Silver: 3,760,806
Total Gold: 13,889

ETL finalizado correctamente.
viernes, 9 de octubre de 2026 8:50:08 p. m.
```

Todo el pipeline (Silver y Gold) se lanza con un único comando: python etl\run_etl.py. Se ejecutó dos veces consecutivas y los conteos finales fueron idénticos (Silver: 3,760,806; Gold: 13,889), lo que demuestra que la carga es idempotente. Esto se logra porque, dentro de una transacción, las tablas se vacían con TRUNCATE y se reconstruyen a partir de los datos Silver en cada ejecución.

- Escriban, sin necesidad de implementarla, la expresión cron (o la configuración del Programador de tareas) que lanzaría ese comando cada día a la medianoche

El pipeline se lanza con un único comando: **python etl/run_etl.py**. Para ejecutarlo todos los días a las 00:00 se usaría la siguiente expresión cron:

*0 0 * * * cd /ruta/al/proyecto && python etl/run_etl.py*

Los cinco campos (minuto, hora, día del mes, mes, día de la semana) indican: minuto 0, hora 0, todos los días del mes, todos los meses y todos los días de la semana. Es decir, a las 00:00 de cada día, según la zona horaria del sistema.

Paso 4 · Power BI

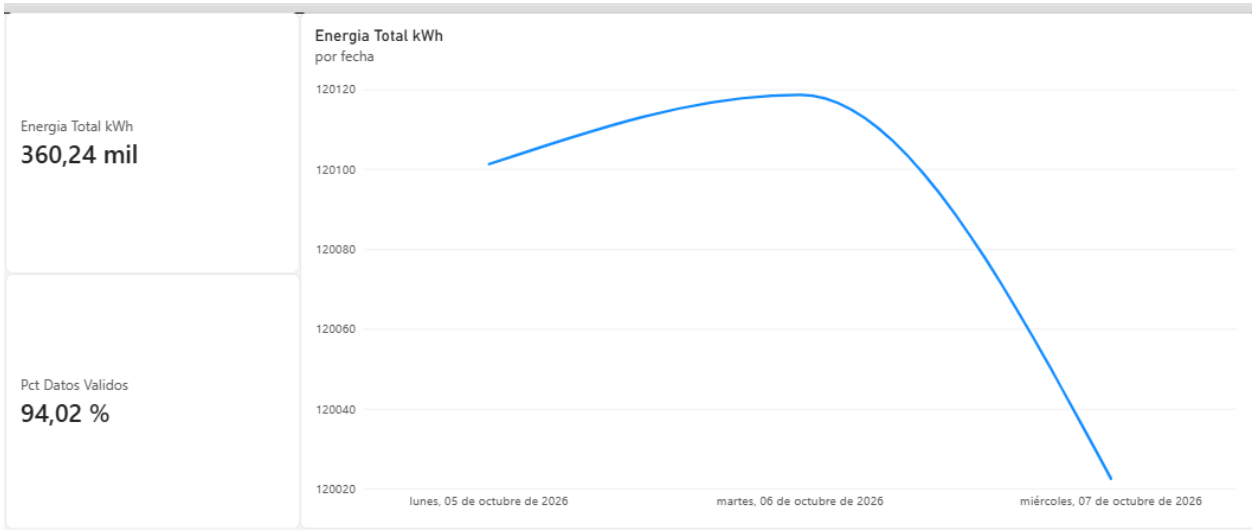
Conecten Power BI Desktop a PostgreSQL y construyan una página con: una tarjeta con la energía total (kWh), un gráfico de líneas con la energía diaria y una tarjeta con el porcentaje de datos válidos. Las cifras deben salir de medidas DAX, no de columnas arrastradas.

Medidas DAX

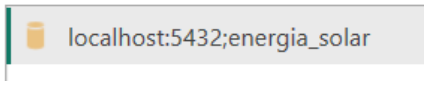
```
Energia Total kWh = SUM('dwh fact_energia_dia'[energia_kwh])
```

```
Pct Datos Validos = DIVIDE(AVERAGE('dwh fact_energia_dia'[pct_datos_validos]), 100)
```

Visuales



Conexión con BD



Paso 5 · Grafana

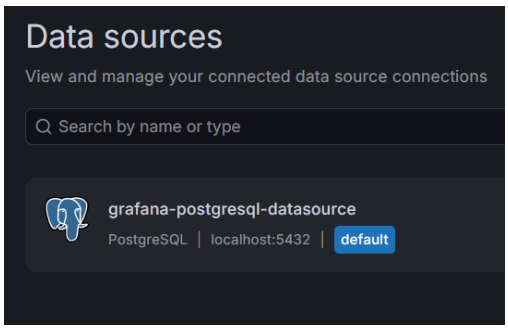
Instalen Grafana OSS en su equipo (instalador o Docker), agreguen PostgreSQL como data source y construyan un dashboard con:

- Un panel de series de tiempo con la potencia `p_ac_kw` cada 5 minutos, usando `$__timeFilter` en la consulta.
- Un umbral (threshold) o una regla de alerta que se active cuando la potencia sea cero entre las 9:00 y las 15:00.

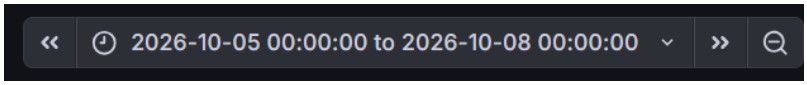
- Un panel tipo stat con la energía del día seleccionado.
- Exporten el dashboard como JSON y guárdenlo en el repositorio.

Si no logran instalar Grafana, documenten el error con capturas, expliquen qué intentaron y exploren los tableros de ejemplo de play.grafana.org para sustentar su comparación. Esta ruta alterna se evalúa con menor puntaje en el criterio de tableros.

Conexion con BD

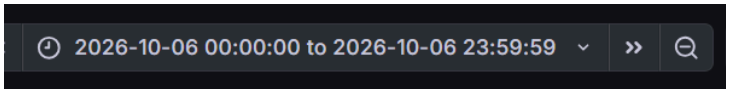


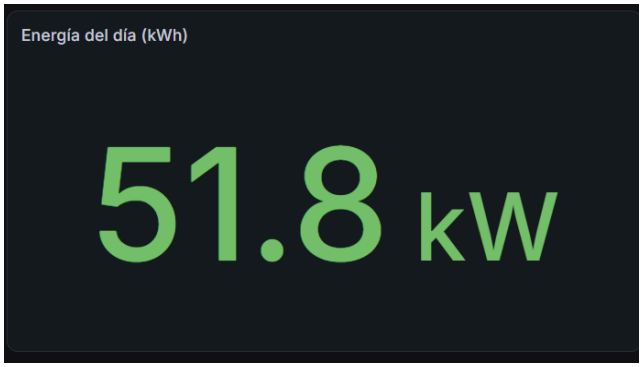
Panel 1





Panel 2





Paso 6 · Reflexión

En máximo 10 líneas: con los mismos datos, ¿qué resolvió mejor cada herramienta?, ¿qué les costó más?, ¿cuál usarían para el operador de la planta y cuál para un directivo?

> **Evidencias de la Parte B en el PDF:** una captura por paso con una explicación de dos o tres líneas, incluidas las capturas de los dos tableros. El código va en el repositorio, no pegado completo en el PDF.

- Qué resolvió mejor cada herramienta: Power BI resolvió mejor el análisis agregado, con medidas DAX para la energía total (360 mil kWh) y el porcentaje de datos válidos (94,02 %). Grafana resolvió mejor el detalle operativo, con la potencia cada 5 minutos y la marca de ceros entre 9:00 y 15:00.
- Qué costó más en Power BI: entender que las cifras debían salir de medidas DAX y no de columnas arrastradas, y ajustar el eje de fechas, que agrupaba por año.
- Qué costó más en Grafana: la configuración inicial (bloqueo de SmartScreen, rango de tiempo y zona horaria UTC), que desplazaba las curvas de potencia.
- Para el operador de la planta: Grafana, por su resolución de 5 minutos, su actualización frecuente y sus umbrales que señalan cuándo un inversor deja de producir.
- Para un directivo: Power BI, por sus indicadores simples, comparables por periodo y de presentación más limpia.
- Conclusión: las dos herramientas se complementan, y ambas dependen de la calidad de las capas Silver y Gold: sin la limpieza previa, las cifras no serían confiables.

5. Parte C · Repositorio en GitHub

Todo el trabajo se desarrolla y se entrega integrado a un repositorio en GitHub. Este repositorio no es de un solo uso: será la base sobre la que continuarán los siguientes trabajos del proyecto hasta el final del semestre.

- Un integrante crea el repositorio **público** con el nombre `solarbi-apellido1-apellido2`.
- Agrega a su compañero como **colaborador** (Settings → Collaborators → Add people). El compañero debe aceptar la invitación antes de la entrega.
- Los dos integrantes deben aportar: mínimo **3 commits por persona**, con mensajes descriptivos (no "update" ni "cambios").
- El `README.md` incluye: nombres completos, curso y grupo, descripción del trabajo, pasos para reproducir la práctica y una tabla breve de quién hizo qué.
- Incluyan un `.gitignore`. **Nunca** suban contraseñas ni cadenas de conexión; usen variables de entorno o un archivo `.env` excluido del repositorio.

Estructura mínima del repositorio:

```
solarbi-apellido1-apellido2/
├── README.md
├── .gitignore
├── docs/       PDF de la consulta
├── data/       bronze/ (crudo) y silver/ (limpio)
├── etl/        simulador.py, run_etl.py
├── sql/        tablas y cargas en PostgreSQL
├── powerbi/    archivo .pbix o proyecto .pbip
└── grafana/    dashboard.json exportado
```

Evidencias que deben aparecer en el PDF: enlace al repositorio, captura de la sección Collaborators con los dos integrantes y captura del historial de commits donde se vean ambos autores.

Enlace del repositorio: https://github.com/sebastianp2200/solarbi-pineda-brinez

6. Condiciones de entrega

- El archivo central es **un único PDF**. La revisión y la calificación se hacen sobre ese PDF: lo que no esté allí no se evalúa.
- El nombre del archivo lleva los **nombres y apellidos completos de los dos integrantes**, así: `NombreCompleto1_NombreCompleto2_Consulta_BI_G01.pdf`.
- Contenido del PDF, en este orden: portada (nombres completos, curso, grupo y fecha), Parte A con cada respuesta identificada por su código, Parte B con las capturas y explicaciones, Parte C con el enlace y las capturas de GitHub, y referencias.
- Los demás archivos (scripts, SQL, datos de muestra y otros que requieran) pueden adjuntarse en la entrega y deben estar también en el repositorio.
- Se sube una sola vez por pareja en Google Classroom, con el enlace al repositorio en el comentario de la entrega.
- Extensión sugerida del PDF: máximo 18 páginas. Mínimo 5 fuentes consultadas, citadas en el texto y listadas al final.

7. Rúbrica de evaluación

Criterio	Descriptor de nivel alto	Peso
Consulta (Parte A)	Respuestas correctas, en palabras propias, aplicadas a SolarBI y con fuentes confiables citadas.	35%
Flujo ETL (Parte B)	Reglas de calidad documentadas, carga idempotente en PostgreSQL con un solo comando y métricas de calidad reportadas.	20%
Tableros (Parte B)	Power BI con medidas DAX y Grafana con consulta de series de tiempo, ambos sobre la misma base, y una reflexión comparativa bien argumentada.	20%
GitHub (Parte C)	Repositorio ordenado, README reproducible, compañero como colaborador y commits de ambos integrantes.	15%
Presentación y ética	PDF con el nombre exigido, bien organizado, con referencias y sin credenciales expuestas.	10%

8. Proyección del trabajo

Esta consulta es el primero de una serie de trabajos que, de aquí al final del semestre, estarán ligados al proyecto SolarBI Pascual. Lo que construyan aquí (repositorio, flujo ETL y datos simulados) se reutiliza en el Entregable III (esquema estrella y flujo ETL) y en el cuadro de mando final.

Con base en estas entregas se identificarán los trabajos con mayor potencial para continuar hacia el PIA y la ruta EnContexto. Trabajen pensando en que su repositorio puede ser el punto de partida de ese proyecto.

9. English corner

Vocabulario técnico de este trabajo: data governance, data steward, data contract, data lineage, single source of truth, row-level security, data source, panel, time series, alert rule, threshold, real-time monitoring, scheduled refresh, incremental load, idempotent, orchestration, broker, topic, payload, version control, commit, pull request.

> Frase para el README: "One governed dataset, two views: Grafana for real-time operations and Power BI for business decisions."

Fuentes

https://grafana.com/docs/grafana/latest/

https://learn.microsoft.com/en-us/power-bi/

https://www.dataversity.net/data-concepts/what-is-the-data-management-body-of-knowledge-dmbok/

https://blog.dataengineerthings.org/the-open-data-contract-standard-is-this-the-missing-interface-for-data-engineering-4372d8c0e26a

https://learn.microsoft.com/en-us/fabric/security/service-admin-row-level-security

https://www.funcionpublica.gov.co/eva/gestornormativo/norma.php?i=49981

https://nibmehub.com/opac-service/pdf/read/The%20Data%20Warehouse%20ETL%20Toolkit%20_%20Practical%20Techniques%20for%20Extracting-%20Cleaning-.pdf

https://www.postgresql.org/docs/current/sql-insert.html

https://learn.microsoft.com/en-us/power-query/query-folding-basics

https://es.scribd.com/document/906472042/ISO-IEC-30141-2024-ed-2-id-88800-Publication-PDF-en

https://www.ibm.com/docs/es/masv-and-l/maximo-monitor/cd?topic=messaging-mqtt-standards-requirements

https://www.influxdata.com/mqtt/

https://0-lucas.github.io/digital-garden/99.-Books/Martin-Kleppmann---Designing-Data-Intensive-Applications_-O%E2%80%99Reilly-Media-(2017).pdf